problems related to mathematics and physics. Let's take a look at one of the problems provided by the SciPy community. The example is available on the Internet (at the SciPy website). However, some of the methods explained in this example are deprecated; hence, we'll rebuild the example, so that it works correctly with the latest version of SciPy and NumPy.

We're going to build and simulate a model based on a coupled spring-mass system, which is essentially a harmonic oscillator, in which a spring is stretched or compressed by a mass, thereby developing a restoring force in the spring, which results in harmonic motions when the mass is displaced from its equilibrium position. For an un-damped system, the motion of Block 1 is given by the following differential equation:

$$m_1 d^2 x_1/dt = (k_1 + k) x_1 - k_2 x_2 = 0$$

For Block 2:

$$m_2 d^2 x_2/dt + k x_2 - k_1 x_1 = 0$$

In this example, we've taken a coupled spring-mass system, which is subjected to a frictional force, thereby resulting in damping. Note that damping tends to reduce the amplitude of oscillations in an oscillatory system. For our example, let us assume that the lengths of the springs, when subjected to no external forces, are *L1* and *L2*. The following differential equations define such a system:

$$m_1 d^2 x_1/dt + \mu_1 dx_1/dt + k_1 (x_1 - L_1) - k_2 (x_1 - x_2 - L_2) = 0$$

And

$$m_2 d^2 x_2/dt + \mu_2 dx/dt + k (x_2 - x_1 - L_2) = 0$$

We'll be using the Scipy *odeint* function to solve this problem. The function works for first-order differential equations; hence, we'll re-write the equations as first fourth order equations:

$$dx_1/dt = y_1$$
$$dy_1/dt = (-\mu_1 y_1 - k_1 (x_1 - L_1) + k (x_2 - x_1 - L_2))/m_1$$
$$dx_2/dt = y$$
$$dy_2/dt = (-\mu_2 y_2 - k_2 (x_2 - x_1 - L_2))/m_2$$

Now, let's write a simple Python script to define this problem:

```
#! /usr/bin/python

def vector(w,t,p):

 x1,y1,x2,y2 = w
 m1,m2,k1,k2,u1,u2,L1,L2 = p

 f = [y1,
        (-b1*y1 - k1*(x1-L1) + k2*(x2-x1-L2))/m1,
         y2,
 (-b2*y2 - k2*(x2-x1-L2))]

 return f
```

In this script, we have simply defined the above mentioned equations programmatically. The argument *w* defines the state variables; *t* is for time, and *p* defines the vector of the parameters. In short, we have simply defined the vector field for the spring-mass system in this script.

Now, let's define a script that uses *odeint* to solve the equations for a given set of parameter values, initial conditions, and time intervals. The script prints the points in the solution to the terminal.

```
#! /usr/bin/python

from scipy.integrate import odeint
import two_springs

# Parameter values
# Masses:
m1 = 1.0
m2 = 1.5
# Spring constants
k1 = 8.0
k2 = 40.0
# Natural lengths
L1 = 0.5
L2 = 1.0
# Friction coefficients
b1 = 0.8
b2 = 0.5

# Initial conditions
# x1 and x2 are the initial displacements; y1 and y2 are the
initial velocities
x1 = 0.5
y1 = 0.0
x2 = 2.25
y2 = 0.0

# ODE solver parameters
abserr = 1.0e-8
relerr = 1.0e-6
stoptime = 10.0
numpoints = 250

# Create the time samples for the output of the ODE solver.
t = [stoptime*float(i)/(numpoints-1) for i in
range(numpoints)]
```